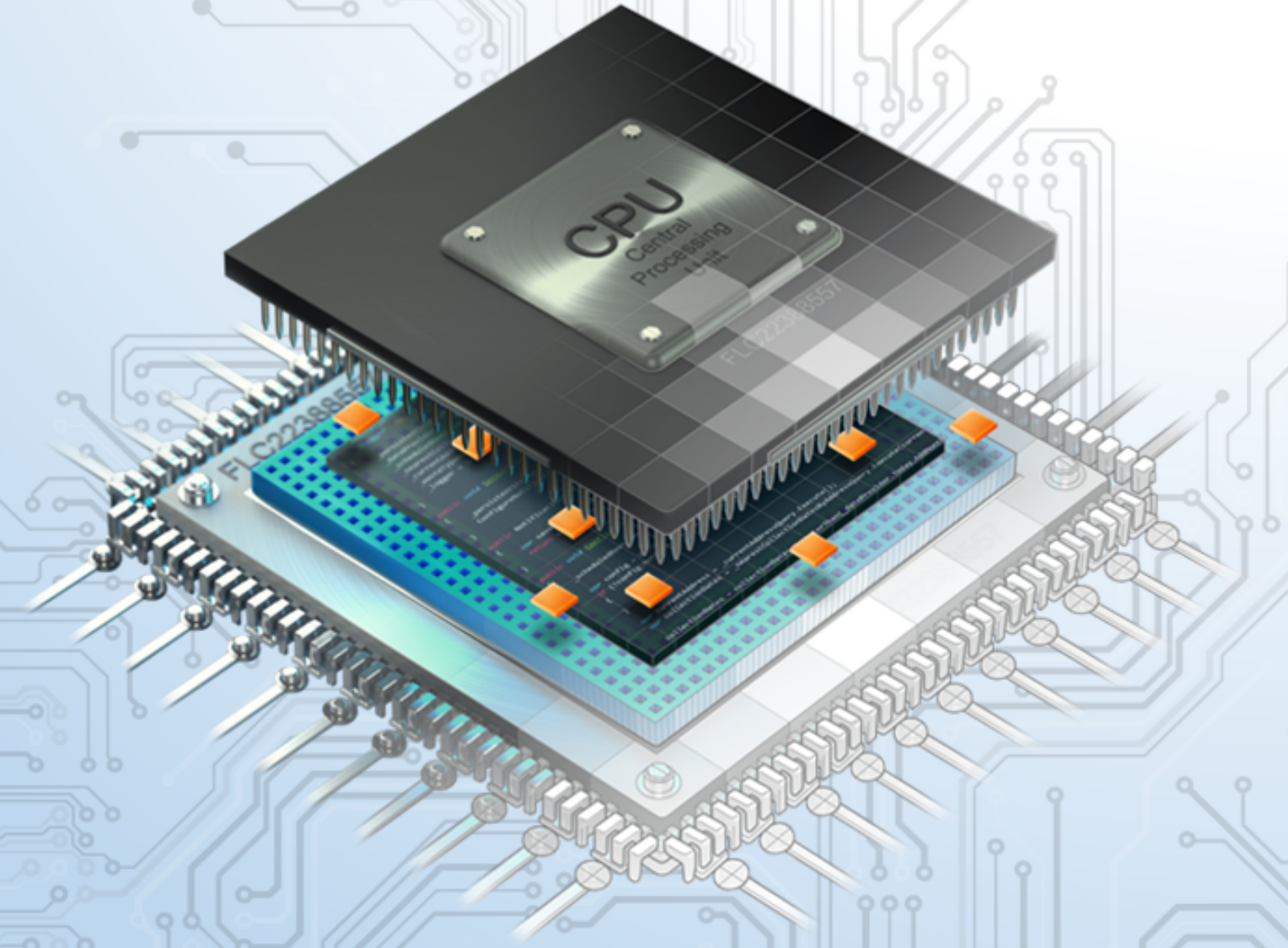
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



Dr. Le Trong Nhan

Mục lục

Chapter 1. Flow and Error Control in Communication **5**

1 Introduction 6

2 Proteus simulation platform 7

3 Project configurations 8

3.1 UART Configuration 8

3.2 ADC Input 9

4 UART loop-back communication 9

5 Sensor reading 10

6 Project description 11

6.1 Command parser 11

6.2 Project implementation 12

7 LAB Report 13

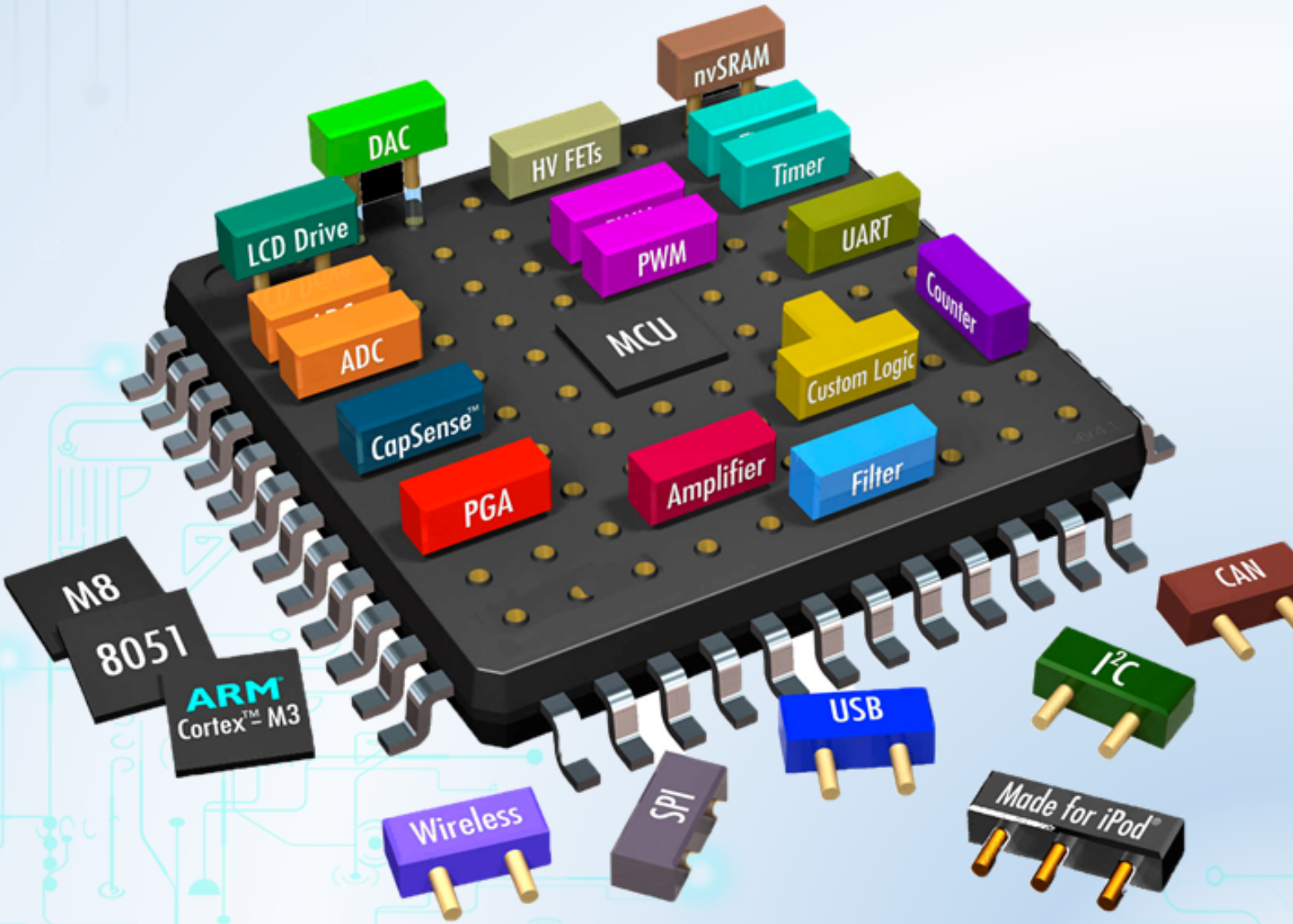
7.1 Present 2 FSM and describe 13

7.2 Implement 2 FSMs in 2 files command_parser.c/.h and command_communication.c/h 14

7.3 Schematic proteus and github repository link 16

CHƯƠNG 1

Flow and Error Control in Communication



1 Introduction

Flow control and Error control are the two main responsibilities of the data link layer, which is a communication channel for node-to-node delivery of the data. The functions of the flow and error control are explained as follows.

Flow control mainly coordinates with the amount of data that can be sent before receiving an acknowledgment from the receiver and it is one of the major duties of the data link layer. For most of the communications, flow control is a set of procedures that mainly tells the sender how much data the sender can send before it must wait for an acknowledgment from the receiver.

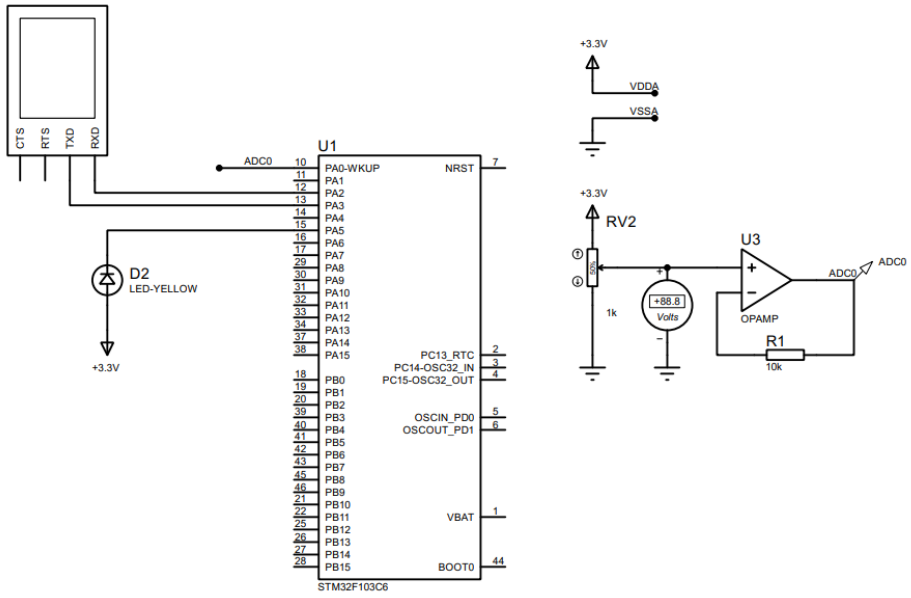
A critical issue, but not really frequently occurred, in the flow control is that the processing rate is slower than the transmission rate. Due to this reason each receiving device has a block of memory that is commonly known as buffer, that is used to store the incoming data until this data will be processed. In case the buffer begins to fill-up then the receiver must be able to tell the sender to halt the transmission until once again the receiver become able to receive.

Meanwhile, error control contains both error detection and error correction. It mainly allows the receiver to inform the sender about any damaged or lost frames during the transmission and then it coordinates with the re-transmission of those frames by the sender.

The term Error control in the communications mainly refers to the methods of error detection and re-transmission. Error control is mainly implemented in a simple way and that is whenever there is an error detected during the exchange, then specified frames are re-transmitted and this process is also referred to as Automatic Repeat request(ARQ).

The target in this lab is to implement a UART communication between the STM32 and a simulated terminal. A data request is sent from the terminal to the STM32. Afterward, computations are performed at the STM32 before a data packet is sent to the terminal. The terminal is supposed to reply an ACK to confirm the communication successfully or not.

2 Proteus simulation platform



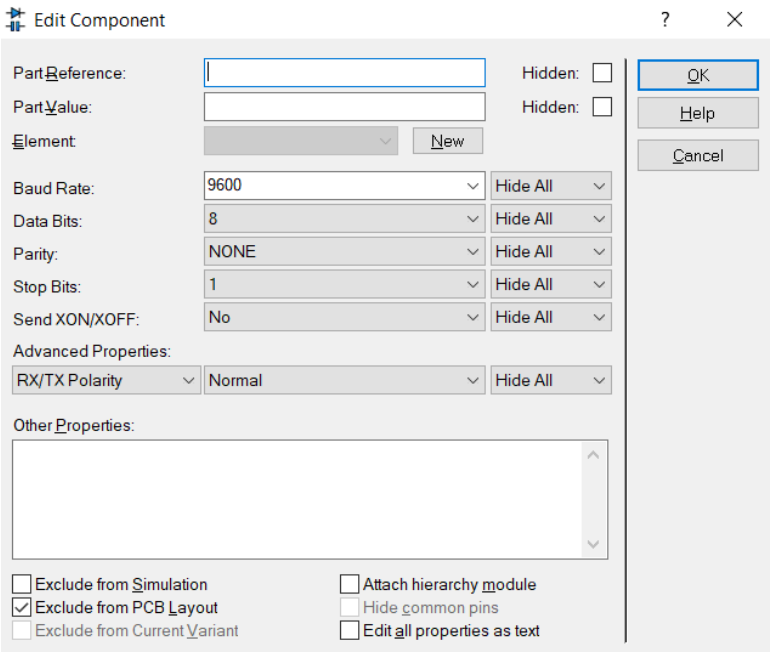
Hình 1.1: *Simulation circuit on Proteus*

Some new components are listed bellow:

- Terminal: Right click, choose Place, Virtual Instrument, then select VIRTUAL TERMINAL.
- Variable resistor (RV2): Right click, choose Place, From Library, and search for the POT-HG device. The value of this device is set to the default 1k.
- Volt meter (for debug): Right click, choose Place, Virtual Instrument, the select DC VOLTMETER.
- OPAMP (U3): Right click, choose Place, From Library, and search for the OPAMP device.

The opamp is used to design a voltage follower circuit, which is one of the most popular applications for opamp. In this case, it is used to design an adc input signal, which is connected to pin PA0 of the MCU.

Double click on the virtual terminal and set its baudrate to 9600, 8 data bits, no parity and 1 stop bit, as follows:



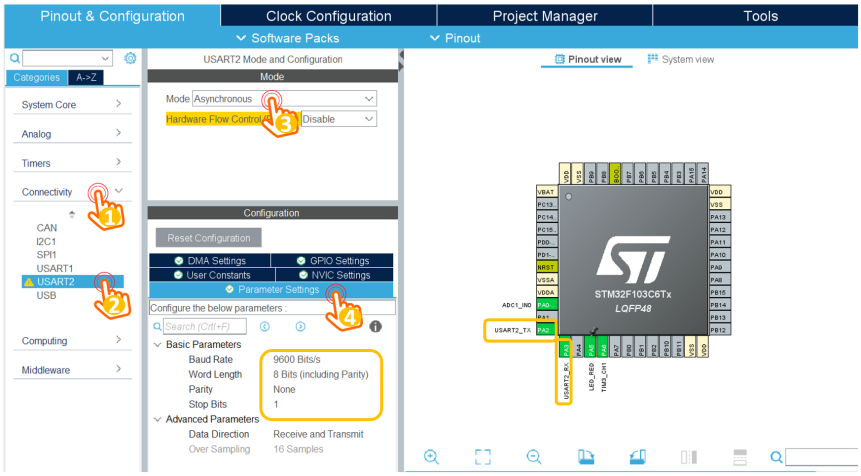
Hình 1.2: Terminal configuration

3 Project configurations

A new project is created with following configurations, concerning the UART for communications and ADC input for sensor reading. The pin PA5 should be an GPIO output, for LED blinky.

3.1 UART Configuration

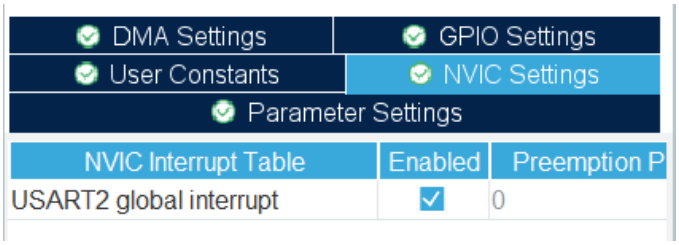
From the ioc file, select **Connectivity**, and then select the **USART2**. The parameter settings for UART channel 2 (USART2) module are depicted as follows:



Hình 1.3: UART configuration in STMCube

The UART channel in this lab is the Asynchronous mode, 9600 bits/s with no Parity and 1 stop bit. After the uart is configured, the pins PA2 (Tx) and PA3(Rx) are enabled.

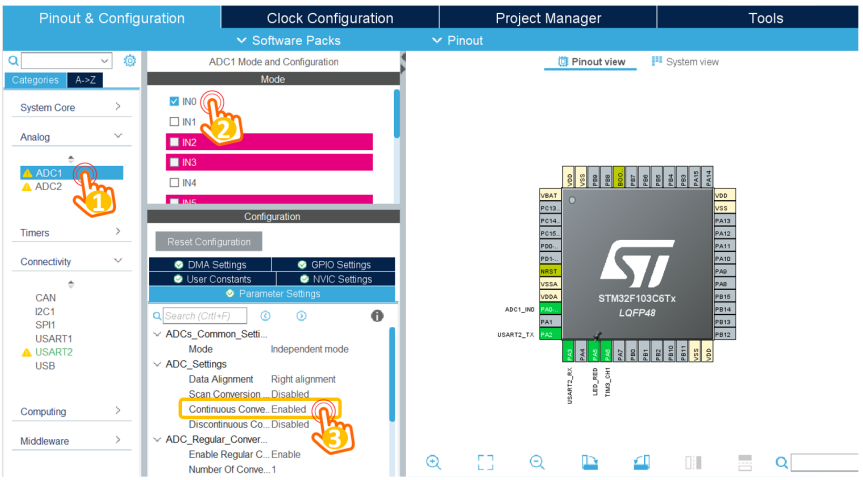
Finally, the NVIC settings are checked to enable the UART interrupt, as follows:



Hình 1.4: *Enable UART interrupt*

3.2 ADC Input

In order to read a voltage signal from a simulated sensor, this module is required. By selecting on **Analog**, then **ADC1**, following configurations are required:



Hình 1.5: *Enable UART interrupt*

The ADC pin is configured to PA0 of the STM32, which is shown in the pinout view dialog.

Finally, the PA5 is configured as a GPIO output, connected to a blinky LED.

4 UART loop-back communication

This source is required to add in the main.c file, to verify the UART communication channel: sending back any character received from the terminal, which is well-known as the loop-back communication.

```c
/* USER CODE BEGIN 0 */
uint8_t temp = 0;

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
  if(huart->Instance == USART2){
    HAL_UART_Transmit(&huart2, &temp, 1, 50);
    HAL_UART_Receive_IT(&huart2, &temp, 1);
  }
}
/* USER CODE END 0 */
```

Program 1.1: Implement the UART interrupt service routine

When a character (or a byte) is received, this interrupt service routine is invoked. After the character is sent to the terminal, the interrupt is activated again. This source code should be placed in a user-defined section.

Finally, in the main function, the proposed source code is presented as follows:

```c
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_ADC1_Init();

  HAL_UART_Receive_IT(&huart2, &temp, 1);

  while (1)
  {
    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
    HAL_Delay(500);
  }

}
```

Program 1.2: Implement the main function

5 Sensor reading

A simple source code to read adc value from PA0 is presented as follows:

```c
uint32_t ADC_value = 0;
while (1)
{
  HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
  ADC_value =  HAL_ADC_GetValue(&hadc1);
```

```
6 HAL_UART_Transmit(&huart2, (void *)str, sprintf(str, "%d\n"
    , ADC_value), 1000);
7   HAL_Delay(500);
8 }
```

Program 1.3: ADC reading from AN0

Every half of second, the ADC value is read and its value is sent to the console. It is worth noticing that the number ADC_value is convert to ascii character by using the sprintf function.

The default ADC in STM32 is 13 bits, meaning that 5V is converted to 4096 decimal value. If the input is 2.5V, ADC_value is 2048.

6 Project description

In this lab, a simple communication protocol is implemented as follows:

- From the console, user types **!RST#** to ask for a sensory data.
- The STM32 response the ADC_value, following a format **!ADC=1234#**, where 1234 presents for the value of ADC_value variable.
- The user ends the communication by sending **!OK#**

The timeout for waiting the **!OK#** at STM32 is 3 seconds. After this period, its packet is sent again. **The value is kept as the previous packet**.

6.1 Command parser

This module is used to received a command from the console. As the reception process is implement by an interrupt, the complexity is considered seriously. The proposed implementation is given as follows.

Firstly, the received character is added into a buffer, and a flag is set to indicate that there is a new data.

```
1 #define MAX_BUFFER_SIZE  30
2 uint8_t temp = 0;
3 uint8_t buffer[MAX_BUFFER_SIZE];
4 uint8_t index_buffer = 0;
5 uint8_t buffer_flag = 0;
6 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
7   if(huart->Instance == USART2){
8
9     //HAL_UART_Transmit(&huart2, &temp, 1, 50);
10    buffer[index_buffer++] = temp;
11    if(index_buffer == 30) index_buffer = 0;
```

```
12
13      buffer_flag = 1;
14      HAL_UART_Receive_IT(&huart2, &temp, 1);
15    }
16  }
```

Program 1.4: Add the received character into a buffer

A state machine to extract a command is implemented in the while(1) of the main function, as follows:

```
1  while (1){
2      if(buffer_flag == 1){
3          command_parser_fsm();
4          buffer_flag = 0;
5      }
6  }
```

Program 1.5: State machine to extract the command

The output of the command parser is to set **command_flag** and **command_data**. In this project, there are two commands, **RTS** and **OK**. The program skeleton is proposed as follows:

```
1  while (1){
2      if(buffer_flag == 1){
3          command_parser_fsm();
4          buffer_flag = 0;
5      }
6      uart_communiation_fsm();
7  }
```

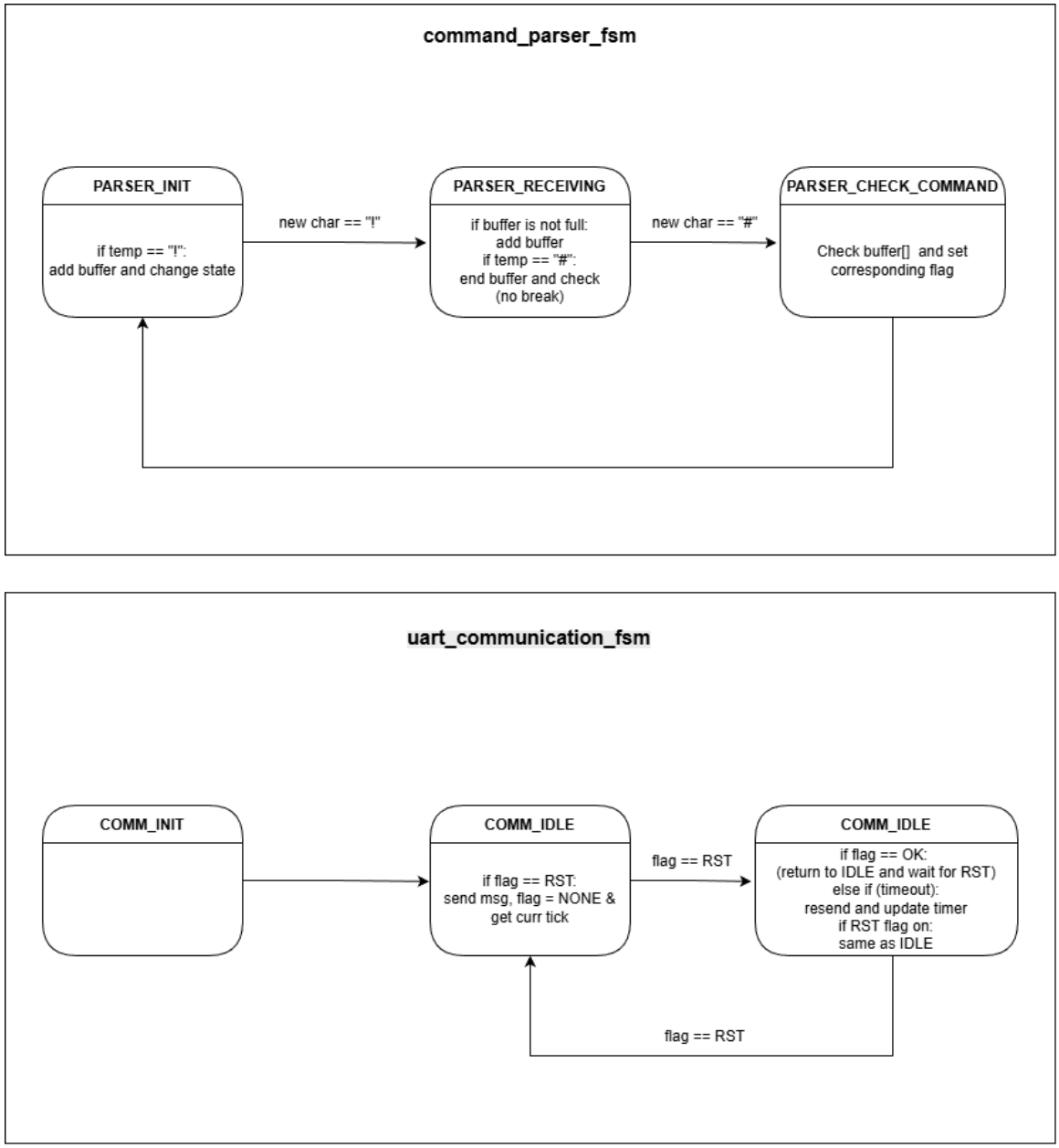
Program 1.6: Program structure

6.2 Project implementation

Students are proposed to implement 2 FSM in seperated modules. Students are asked to design the FSM before their implementations in STM32Cube.

7 LAB Report

7.1 Present 2 FSM and describe



Hình 1.6: FSM for 2 parsing streams and uart communication of the project

Detailed description of FSM:

- command_parser_fsm(): has the function of recording input string from terminal. Add new character to buffer[] starting from character "!" and change state PARSER_RECEIVING. In the new state, record input characters until encountering character "#" or exceeding buffer limit. Immediately check each character in buffer to match **!RST#** or **!OK#** when the new character entered is "#", turn on the corresponding flag and return to PARSER_INIT.
- uart_communication_fsm(): has the function of communicating terminal via uart. COMM_INIT is the state where buffer does not perform task and switches to COMM_IDLE. In this state, check if RST flag is turned on or not? If enabled, display ADC_value, reset flag and get current time (used to calculate timeout) and change state COMM_WAITING, if not enabled, continue waiting. COMM_WAITING state, handle 3 conditions in turn: wait for OK flag to turn on, timeout and re-enter **!RST#**. If flag == OK, return to COMM_IDLE and flag = NONE, restart waiting process **!RST#**. In case of timeout, resend last_msg and update timer for next timeout. If user continues to enter **!RST#**, process similarly to COMM_IDLE but do not change state.

7.2 Implement 2 FSMs in 2 files command_parser.c/.h and command_communication.c/h

```
void command_parser_fsm(void) {
    uint8_t c = temp;
    switch(parser_state) {
    case PARSER_INIT:
        if (c == '!')
        {
            index_buffer = 0;
            buffer[index_buffer++] = c;
            parser_state = PARSER_RECEIVING;
        }
        break;
    case PARSER_RECEIVING:
        if (index_buffer < MAX_BUFFER_SIZE - 1)
        {
             buffer[index_buffer++] = c;
        }
        if (c == '#')
    {
      buffer[index_buffer] = '\0';
      parser_state = PARSER_CHECK_COMMAND;
    }
    if (parser_state != PARSER_CHECK_COMMAND)
    {
      break;
    }
```

```
    case PARSER_CHECK_COMMAND:
        if (strcmp((char*)buffer, "!RST#") == 0)
        {
            flag = CMD_RST;
        }
        else if (strcmp((char*)buffer, "!OK#") == 0)
        {
            flag = CMD_OK;
        } else {
            flag = CMD_NONE;
        }
        parser_state = PARSER_INIT;
      break;
    default:
        parser_state = PARSER_INIT;
        break;
    }
}
```

Program 1.7: command_parser_fsm() implementation in command_parser.c/.h

```
void send_uart(char *str) {
    HAL_UART_Transmit(&huart2, (uint8_t*)str, strlen(str),
   1000);
}

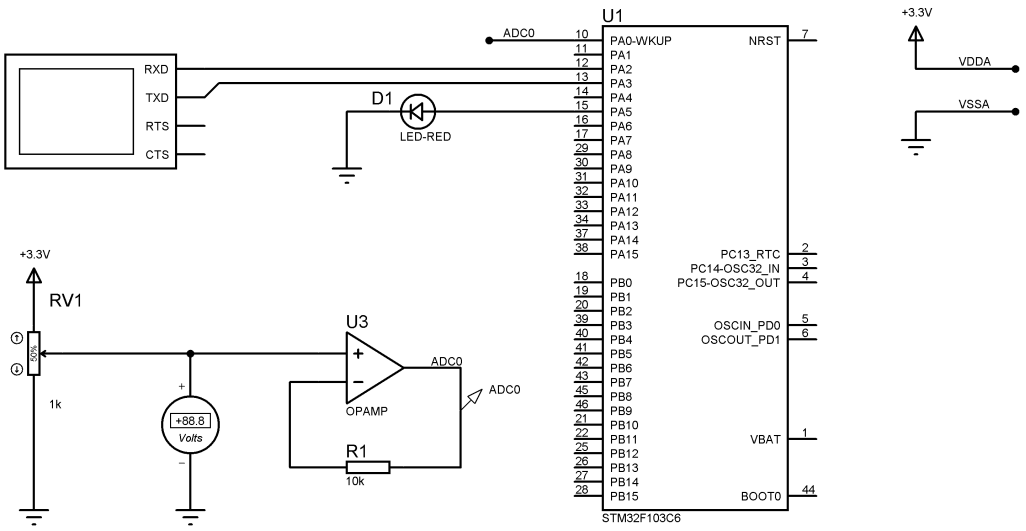
void uart_communication_fsm(void) {
    switch (comm_state) {
    case COMM_INIT:
        comm_state = COMM_IDLE;
        break;
    case COMM_IDLE:
        if (flag == CMD_RST)
        {
            sprintf(last_msg, "\r\n!ADC=%d#", ADC_value);
            send_uart(last_msg);
            timer = HAL_GetTick();
            comm_state = COMM_WAITING;
            flag = CMD_NONE;
        }

        if (flag == CMD_OK)
        {
            flag = CMD_NONE;
        }
        break;
    case COMM_WAITING:
        if (flag == CMD_OK)
        {
            flag = CMD_NONE;
```

```
            timer = 0;
            comm_state = COMM_IDLE;
            break;
        }
        else if (HAL_GetTick() - timer >= 3000)
        {
            send_uart(last_msg);
            timer = HAL_GetTick();
        }
        else if (flag == CMD_RST)
        {
            sprintf(last_msg, "\r\n!ADC=%d#", ADC_value);
            send_uart(last_msg);
            timer = HAL_GetTick();
            flag = CMD_NONE;
        }
        break;
    default:
      break;
    }
}
```

Program 1.8: uart_communication_fsm() implementation in command_communication.c/.h

7.3 Schematic proteus and github repository link



Hình 1.7: FSM for 2 parsing streams and uart communication of the project

Github: https://github.com/Nguyen-Hao-Khang/Microprocessor-Microcontroller-LABs-HCMUT-23
git